ARTIFICIAL INTELLIGENCE (CT 653)

Tribhuvan University | Institute of Engineering | BCT | Year III / Part II

CHAPTER 3: Solving Problems by Searching

Syllabus Hours: 10 hrs | Marks Weight: 16 marks

Complete Exam-Ready Answers | All Past Questions Covered (2070–2082)

CHAPTER 3 SYLLABUS TOPICS

- **3.1** Problem-Solving Agents
- **3.2** Searching for Solutions — Problem Formulation
- **3.3** Uninformed (Blind) Search Strategies
- **3.4** Informed (Heuristic) Search Strategies
- **3.5** Local Search and Optimization Problems
- **3.6** Adversarial Search — Game Playing

SECTION 1: Problem-Solving Agents & Problem Formulation

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q1. Define problem-solving agent. What are the components of a well-defined problem? Explain the 8-puzzle problem as a search problem.	**[2+4+2]** *2081 Chaitra 2079 Chaitra 2075 Bhadra*

Answer: Problem-Solving Agent

A problem-solving agent is a goal-based agent that plans ahead — it considers a sequence of actions to reach a desired goal state. Instead of reacting immediately to the current percept, it builds an internal model of the world, formulates a goal, and searches for a sequence of actions that leads to that goal.

Step	Agent Activity
Goal Formulation	Define the desired end state the agent wants to achieve.
Problem Formulation	Define the states, actions, initial state, goal test, and path cost.
Search	Find a sequence of actions leading from initial to goal state.
Execution	Carry out the solution action by action.

Components of a Well-Defined Problem (5 components):

Component	Description	8-Puzzle Example
Initial State	The starting configuration/state of the problem.	A 3×3 grid with tiles 1–8 arranged in a specific scrambled order, with one blank tile.
Actions (Operators)	The set of all possible actions the agent can perform in a given state.	Move blank tile UP, DOWN, LEFT, or RIGHT (where the move is valid — within grid bounds).
Transition Model / Result	Defines the state that results from performing an action in a state. Together with actions, defines the state space.	Sliding a tile into the blank position produces a new board configuration.
Goal Test	A function that checks whether a given state is the goal state.	All tiles 1–8 arranged in order with blank in bottom-right corner.
Path Cost (Step Cost)	A function that assigns a cost to each path/action sequence. Typically we want the lowest-cost solution.	Each tile move costs 1. Total cost = total number of moves made.

8-Puzzle as a Search Problem:

The 8-puzzle consists of a 3×3 grid with 8 numbered tiles and one blank space. The agent must slide tiles to reach the goal configuration. The state space has 9!/2 = 181,440 reachable states.

	Initial State	Goal State
Grid	7 2 4 | 5 _ 6 | 8 3 1	1 2 3 | 4 5 6 | 7 8 _
Representation	3×3 integer array; blank = 0	1 2 3 / 4 5 6 / 7 8 0

- State space size: 9! = 362,880 total configurations (181,440 reachable).
- Branching factor: ~2.67 average (blank at corner=2, edge=3, center=4 moves).
- Optimal solution depth: typically 20–26 moves for random initial state.
- Heuristics used: h1 = number of misplaced tiles; h2 = Manhattan distance (sum of tile distances from goal).

Q2. List the characteristics of a well-defined problem. Give examples of problems that can be solved by search and formulate them.	**[2+6]** *2081 Ashwin 2078* *Chaitra 2074 Bhadra*

Answer: Well-Defined Problems and Formulation Examples

Characteristics of a Well-Defined Problem:

- 1. The initial state is clearly specified.
- 2. The set of all possible actions (operators) is defined for each state.
- 3. The transition model (result of each action) is known.
- 4. The goal test can be applied to any state to check if it is the goal.
- 5. A path cost function assigns a numerical value to each action sequence.
- 6. The state space is finite or well-structured enough to search.

Classic Search Problem Formulations:

Problem	Initial State	Actions	Goal Test	Path Cost
Vacuum World	Dirty floor in room A; agent in A	Move Left, Move Right, Suck	All rooms clean	1 per action
8-Queens	Empty 8×8 chessboard	Place queen in any column	8 queens placed, none attacking	0 (find any solution)
Missionaries & Cannibals	3M + 3C on left bank; boat on left	Move 1–2 people across river	All 6 on right bank; never outnumbered	1 per crossing
Route Finding	Source city (e.g. Kathmandu)	Drive to adjacent city	Reach destination city	Road distance / travel time
N-Queens (N=any)	Empty N×N board	Place queen in next row	N queens, no two attacking	0 (find any solution)
Water Jug	(0,0) two empty jugs (4L, 3L)	Fill, empty, pour between jugs	One jug has exactly 2L	1 per action

SECTION 2: Uninformed (Blind) Search Strategies

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q3. What is uninformed search? Explain Breadth-First Search (BFS) and Depth-First Search (DFS) with algorithm, example, and evaluation using the criteria: completeness, optimality, time complexity, and space complexity.

[2+6]
2081 Chaitra 2079 Chaitra 2076 Baisakh

Answer: Uninformed Search

Uninformed (blind) search strategies have no additional information about states beyond the problem definition — they do not know how far they are from the goal. They only know the initial state, actions, transition model, and goal test. Examples include BFS, DFS, UCS, DLS, and IDDFS.

Breadth-First Search (BFS):

BFS explores all nodes at the current depth level before going deeper. It uses a FIFO queue as the frontier (open list).

Step	Action	Description
1	**Initialize**	Add initial state to FIFO queue (frontier). Mark as visited.
2	**Dequeue**	Remove the front node from the queue.

3	**Goal Test**	If current node is goal state, return path. STOP.
4	**Expand**	Generate all unvisited successors/children of current node.
5	**Enqueue**	Add all successors to the BACK of the queue. Mark as visited.
6	**Repeat**	Go to Step 2. If queue is empty and goal not found, return FAILURE.

DFS — Depth-First Search:

DFS explores as deep as possible along each branch before backtracking. It uses a LIFO stack (or recursion) as the frontier.

Step	Action	Description
1	**Initialize**	Push initial state onto LIFO stack. Mark as visited.
2	**Pop**	Remove the TOP node from the stack.
3	**Goal Test**	If current node is goal state, return path. STOP.
4	**Expand**	Generate all unvisited successors of current node.
5	**Push**	Push all successors onto the TOP of the stack. Mark as visited.
6	**Repeat**	Go to Step 2. If stack is empty and goal not found, return FAILURE.

BFS vs DFS — Evaluation:

Criterion	BFS	DFS
Completeness	YES — will always find a solution if one exists (in finite graphs).	NO — may get trapped in infinite branches. Incomplete on infinite state spaces.
Optimality	YES — finds shallowest (least-depth) solution. Optimal if all step costs are equal.	NO — may find a deep, non-optimal solution first.
Time Complexity	O(b^d) — b = branching factor, d = depth of shallowest goal.	O(b^m) — m = maximum depth of the search tree.
Space Complexity	O(b^d) — keeps ALL nodes in memory (very high).	O(b·m) — only keeps nodes on the current path (very low). Main advantage of DFS.
Data Structure	FIFO Queue (First-In, First-Out)	LIFO Stack (Last-In, First-Out) or Recursion

Best When	Solution is at shallow depth; completeness required.	Solution is known to be deep; memory is limited.

Most Repeated: "Explain BFS and DFS with algorithm. Compare their performance." is the most repeated question in this chapter. Always show the algorithm steps + the evaluation criteria table.

Q4. What is Uniform Cost Search (UCS)? How does it differ from BFS? Also explain Depth-Limited Search (DLS) and Iterative Deepening DFS (IDDFS) with their properties.

[2+5]
2080 Ashwin 2078
Baishakh 2074 Bhadra

Answer: UCS, DLS and IDDFS

Uniform Cost Search (UCS):

UCS expands the node with the lowest total path cost g(n). It uses a Priority Queue (min-heap) ordered by path cost. UCS generalizes BFS — when all step costs are equal, UCS = BFS.

Step	Action	Description
1	**Initialize**	Add initial state to Priority Queue with cost = 0.
2	**Dequeue Min**	Remove the node with the LOWEST path cost from the priority queue.
3	**Goal Test**	If current node is goal, return path with its cost.
4	**Expand**	Generate all successors. For each, compute g(child) = g(parent) + step_cost.
5	**Insert**	Add successors to priority queue with updated path costs.
6	**Repeat**	Go to Step 2. Return FAILURE if queue empty.

Depth-Limited Search (DLS):

DLS is DFS with a depth cutoff limit L. Nodes at depth L are not expanded. This prevents infinite looping in infinite state spaces.

- Algorithm: Run DFS, but do not expand any node at depth ≥ L.
- Completeness: NO — may fail if goal is deeper than L. Returns failure if no solution found within limit.
- Optimality: NO.
- Time: O(b^L), Space: O(b·L).
- Problem: Choosing L is hard — too small misses goals, too large wastes time.

Iterative Deepening Depth-First Search (IDDFS):

IDDFS combines the best of BFS (completeness, optimality) and DFS (low memory). It runs DLS repeatedly with increasing depth limit L = 0, 1, 2, 3, ... until the goal is found.

Step	Action	Description
1	**Set L = 0**	Initialize depth limit to 0.
2	**DLS(L)**	Run Depth-Limited Search with current limit L.
3	**Goal Found?**	If DLS returns a solution, return it. STOP.
4	**Increment**	L = L + 1. Go to Step 2.
5	**Terminate**	If state space exhausted at all depths, return FAILURE.

Property	UCS	DLS	IDDFS
Completeness	YES (if costs > 0)	NO (if L < goal depth)	YES (complete like BFS)
Optimality	YES (always finds min cost)	NO	YES (if step costs equal)
Time	O(b^(C*/ε))	O(b^L)	O(b^d)
Space	O(b^(C*/ε))	O(b·L)	O(b·d) — very low
Data Structure	Priority Queue	Stack with depth check	Stack (depth reset each iter.)
Best When	Costs vary between actions	Upper bound on depth known	Optimal but memory-limited

Note: C* = optimal path cost; ε = minimum step cost. IDDFS is the preferred uninformed search method in practice because it achieves BFS-level optimality with DFS-level memory usage.

Q5. Compare all uninformed search strategies with respect to completeness, optimality, time complexity, and space complexity. Which one would you use and when?

[2+6]

2079 Ashwin Back
2077 Chaitra 2073 Bhadra

Answer: Comparison of All Uninformed Search Strategies

Strategy	Complete?	Optimal?	Time	Space	Best Use Case
BFS	Yes	Yes*	O(b^d)	O(b^d)	Shallowest goal; memory available
DFS	No	No	O(b^m)	O(b·m)	Deep goals; memory limited

UCS	Yes	Yes	O(b^C*/ε)	O(b^C*/ε)	Variable step costs
DLS (limit L)	No	No	O(b^L)	O(b·L)	Known depth bound
IDDFS	Yes	Yes*	O(b^d)	O(b·d)	Best general uninformed
Bidirectional BFS	Yes	Yes*	O(b^d/2)	O(b^d/2)	Known goal; reduce complexity

*BFS / IDDFS optimal only when all step costs are equal. UCS is optimal for any positive step costs.

b = branching factor | d = depth of shallowest goal | m = max depth | C* = optimal cost | ε = min step cost

SECTION 3: Informed (Heuristic) Search Strategies

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2075 Baisakh, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2070 Magh

Q6. What is an informed search? Define heuristic function. Explain Greedy Best-First Search and A* search with algorithms and examples. What makes A* optimal?

[2+6]
2081 Chaitra 2079 Chaitra 2075 Bhadra

Answer: Informed Search & Heuristic Functions

Informed search strategies use problem-specific knowledge beyond the problem definition to guide the search more efficiently. This knowledge is encoded in a heuristic function h(n).

Heuristic Function h(n):

A heuristic function h(n) estimates the cost from node n to the nearest goal state. It provides guidance about which direction looks most promising without computing the exact cost.

- h(n) = 0 if n is a goal state.
- h(n) ≥ 0 for all n (non-negative).
- Example: In route finding, h(n) = straight-line distance from city n to destination.
- Example: In 8-puzzle, h1 = number of misplaced tiles; h2 = sum of Manhattan distances.

Greedy Best-First Search:

Greedy Best-First Search expands the node that appears closest to the goal according to h(n). It uses a priority queue ordered by h(n) alone.

- Evaluation function: f(n) = h(n)
- Completeness: NO — can get trapped in loops or infinite branches.
- Optimality: NO — may find a suboptimal path.

- Time: O(b^m) in worst case, but often much better in practice.
- Space: O(b^m) — keeps all nodes in memory.
- Example: In Romania route-finding, greedy always goes to the city with smallest straight-line distance to Bucharest.

A* Search (A-Star):

A* is the most widely used informed search algorithm. It combines the actual path cost g(n) with the heuristic h(n) to evaluate nodes.

Component	Meaning
g(n)	Actual cost from the initial state to node n (known, exact value).
h(n)	Estimated cost from node n to the nearest goal (heuristic, approximate).
f(n) = g(n) + h(n)	Estimated total cost of the cheapest solution through node n. A* always expands the node with lowest f(n).

Step	Action	Description
1	**Initialize**	Add start node to Priority Queue with f(start) = g(start) + h(start). g(start)=0.
2	**Dequeue**	Remove node n with lowest f(n) from the priority queue.
3	**Goal Test**	If n is the goal, return the path. STOP.
4	**Expand**	Generate all successors s of n. For each: g(s) = g(n) + step_cost(n,s); f(s) = g(s) + h(s).
5	**Update**	If s is in frontier with higher f, replace. If s is in explored set, skip. Otherwise add s to frontier.
6	**Repeat**	Go to Step 2. Return FAILURE if frontier is empty.

Why A* is Optimal — Admissibility and Consistency:

Property	Definition	Importance
Admissibility	h(n) never overestimates the true cost to the goal. h(n) ≤ h*(n) for all n, where h*(n) is the actual cost.	Guarantees A* finds the OPTIMAL solution when searching tree.
Consistency (Monotonicity)	h(n) ≤ c(n,a,n') + h(n') for every node n and successor n' via action a. (Triangle inequality)	Guarantees no node needs to be re-expanded — A* on graphs is optimal.
Dominance	h2 dominates h1 if h2(n) ≥ h1(n) for all n. Both must be admissible.	A higher (but admissible) heuristic expands fewer nodes — more efficient.

- h1 (misplaced tiles) is admissible because each misplaced tile requires at least 1 move.
- h2 (Manhattan distance) is admissible and dominates h1 — therefore better for 8-puzzle.

A* Example — Romania Route Finding (Arad → Bucharest):

Iteration	Node Expanded	f(n)=g(n)+h(n)	Path So Far
1	Arad (start)	0+366=366	Arad
2	Sibiu	140+253=393	Arad→Sibiu
3	Rimnicu Vilcea	220+193=413	Arad→Sibiu→Rimnicu
4	Fagaras	239+176=415	Arad→Sibiu→Fagaras
5	Pitesti	317+100=417	Arad→Sibiu→Rimnicu→Pitesti
6	Bucharest (GOAL)	418+0=418	Arad→Sibiu→Rimnicu→Pitesti→Bucharest

Optimal cost found: 418 km. BFS/Greedy would find suboptimal paths. A* found the true shortest route.

Q7. What are admissible and consistent heuristics? Compare A* and Greedy Best-First Search. What is AO* algorithm and how does it differ from A*?

[2+4+2]

2079 Ashwin Back 2077 Chaitra 2072 Ashwin

Answer: Heuristics, A* vs Greedy, and AO*

A* vs Greedy Best-First Search — Comparison:

Feature	A* Search	Greedy Best-First Search
Evaluation Function	f(n) = g(n) + h(n) — uses both actual and estimated cost.	f(n) = h(n) — uses only estimated cost to goal.
Completeness	YES (with finite state space, or closed list).	NO — may loop on infinite spaces.
Optimality	YES — if h(n) is admissible (tree) or consistent (graph).	NO — often finds suboptimal solution.
Time Complexity	Exponential in worst case; depends heavily on heuristic quality.	Can be very fast in practice; polynomial with good heuristic.
Space Complexity	Keeps all generated nodes in memory — O(b^d).	Also O(b^m) — same memory issue.

Behavior	More systematic; considers cost already paid.	More direct/"greedy"; may take wrong shortcuts.
Analogy	Uses both GPS distance and mileage already driven.	Only uses GPS distance to destination.

AO* Algorithm (And-Or Search):

AO* is an extension of A* designed for AND-OR graphs — problems where a solution may require solving ALL sub-problems (AND nodes) or only ONE of several alternatives (OR nodes). It is used for problem decomposition.

- OR node: The agent can choose any ONE successor to solve — like regular search.
- AND node: The agent must solve ALL successors — subproblems that ALL must be satisfied.
- AO* evaluates cost as the sum of all AND-successors' costs, vs minimum of OR-successors' costs.
- AO* finds the minimum-cost solution graph, not just a solution path.

Feature	A*	AO*
Graph Type	Regular directed graphs (OR trees)	AND-OR graphs (problem decomposition)
Search Goal	Finds a solution PATH	Finds a solution GRAPH (tree of sub-solutions)
Node Type	All nodes are OR nodes (choose one action)	Both AND nodes (must solve all) and OR nodes
Application	Route finding, 8-puzzle, game playing	Theorem proving, expert systems, task planning
Backtracking	Limited — explores forward	Full backtracking through AND-OR structure

SECTION 4: Local Search Algorithms and Optimization

Exam Ref: 2082 Kartik, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2079 Ashwin Back, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra

Q8. What are local search algorithms? Explain Hill Climbing with its types, limitations, and ways to overcome them.

[2+6]
2081 Ashwin 2079 Chaitra 2075 Bhadra

Answer: Local Search & Hill Climbing

Local search algorithms operate on a single current state (not a systematic search of the full space). They move to neighboring states that improve an objective function. They are memory-efficient (O(1) space) but do not guarantee a globally optimal solution. Used when the path to the goal does not matter — only the final state/configuration.

Feature	Systematic Search (BFS/DFS/A*)	Local Search
Memory	High — stores frontier/explored	Very low — only current state (O(1))
Solution	Complete path from start to goal	Final state only (path irrelevant)
Goal	Find goal state	Find optimal state (maximize/minimize objective)
Completeness	Often complete	Not complete (may get stuck)
Examples	BFS, DFS, A*, UCS	Hill Climbing, SA, Genetic Algorithm

Hill Climbing (Greedy Local Search):

Hill Climbing is a local search that continuously moves in the direction of increasing value (gradient ascent). It stops when no neighbor has a higher value than the current state — a peak is reached.

Step	Action	Description
1	**Initialize**	Start at a random initial state.
2	**Evaluate Neighbors**	Generate all successors of current state and evaluate their objective function values.
3	**Select Best**	Choose the best-valued neighbor (highest for maximization; lowest for minimization).
4	**Move?**	If best neighbor > current state → move to best neighbor (go to Step 2).
5	**Stop**	If no neighbor improves the current state → STOP (local maximum found).

Types of Hill Climbing:

Type	Description	Property
Simple Hill Climbing	Moves to the FIRST neighbor found that is better than current state.	Fast per step; may miss better neighbors.
Steepest-Ascent HC	Evaluates ALL neighbors and moves to the BEST one.	Slower per step; more informed choice.
Stochastic HC	Randomly chooses among uphill moves (not always the steepest).	Avoids some local maxima; randomized.

Random-Restart HC	Restarts from a random state when stuck. Repeats multiple times.	Can escape local maxima by restarting.
Sideways Moves HC	Allows moves to a neighbor with EQUAL value (lateral moves).	Handles plateaus but may loop.

Limitations (Problems) of Hill Climbing:

Problem	Description	Example
Local Maximum	A state that is better than all neighbors but NOT the global optimum. Stuck here.	In 8-queens: some arrangements cannot be improved but still have conflicts.
Plateau	A flat region of the state space where all neighbors have the same value. No uphill move.	Many states with the same h value in 8-puzzle.
Ridge	A peak that is steep in one direction — standard moves cannot improve it; diagonal needed.	Diagonal peaks in optimization landscapes.
Global Optimum not guaranteed	Hill climbing finds LOCAL optima, not guaranteed to find the GLOBAL optimum.	Random restarts needed to improve probability of finding global max.

Solutions to Hill Climbing Problems:

- Random-Restart Hill Climbing: Restart from multiple random initial states. The best solution found across all restarts is kept.
- Simulated Annealing: Allow occasional downhill moves with a probability that decreases over time.
- Genetic Algorithms: Use population of states with crossover and mutation to escape local optima.
- Tabu Search: Maintain a list of recently visited states to avoid revisiting them (escape cycles).

Q9. What is Simulated Annealing? How does it overcome the limitations of hill climbing? Compare Hill Climbing with Simulated Annealing.	**[2+4+2]** *2080 Ashwin 2078 Chaitra 2073 Bhadra*

Answer: Simulated Annealing

Simulated Annealing (SA) is a local search algorithm inspired by the metallurgical process of annealing — slowly cooling a metal to minimize its energy (defects). SA escapes local maxima by occasionally accepting worse moves with a probability that decreases over time (temperature schedule).

Step	Action	Description
1	**Initialize**	Set current state = random initial state. Set temperature T = T_max (high initial temperature).

2	**Pick Neighbor**	Randomly select a neighbor state n of the current state.
3	**Compute ΔE**	ΔE = Value(n) − Value(current). (ΔE > 0 means n is better.)
4	**Accept?**	If ΔE > 0: always move to n. If ΔE ≤ 0: accept n with probability P = e^(ΔE/T).
5	**Cool Down**	Reduce temperature: T = α × T (where α is cooling rate, 0 < α < 1).
6	**Terminate**	Stop when T ≈ 0 (frozen) or a good enough solution is found.

Key insight: At high temperature, SA behaves like random search (accepts bad moves freely). As T decreases, it becomes more like hill climbing (rarely accepts bad moves). This allows SA to escape local maxima early and converge to a global optimum late.

Feature	Hill Climbing	Simulated Annealing
Accepting bad moves	NEVER — always moves uphill only.	YES — accepts bad moves with probability e^(ΔE/T).
Escaping local maxima	Cannot — gets permanently stuck at local optima.	Can — bad moves allow escape early in the process.
Optimality	Not guaranteed (finds local optimum).	Theoretically finds global optimum if T cooled slowly enough.
Randomness	Deterministic (given fixed neighbors).	Stochastic — random neighbor selection AND random acceptance.
Temperature	No temperature parameter.	Uses temperature schedule (cooling rate α).
Speed	Fast (no probability computation).	Slower due to probability calculations and cooling schedule.
Application	Quick approximate solutions, time-critical.	Optimization: VLSI design, TSP, scheduling, protein folding.

Q10. What is a Genetic Algorithm (GA)? Explain the steps of GA with an example. How does GA differ from Hill Climbing?

[2+5+1]

2079 Ashwin Back
2077 Chaitra 2071 Bhadra

Answer: Genetic Algorithms

A Genetic Algorithm (GA) is a local search/optimization technique inspired by biological evolution — natural selection, crossover (recombination), and mutation. GA maintains a POPULATION of candidate solutions and evolves them over generations toward better solutions.

GA Terminology:

Term	Meaning	Example (8-Queens)
Individual / Chromosome	A single candidate solution — encoded as a string of genes.	A permutation like [3,1,6,2,8,5,7,4] representing queen positions.
Population	A set of individuals (current generation of solutions).	A set of 100 random queen arrangements.
Fitness Function	Measures how "good" a solution is. Higher = better.	Number of non-attacking queen pairs (max = 28 for 8-queens).
Selection	Choose individuals for reproduction — fitter ones have higher chance.	Roulette wheel selection, tournament selection.
Crossover (Recombination)	Combine two parent chromosomes to produce offspring.	Swap portions of two parent queen permutations at a crossover point.
Mutation	Randomly alter a gene in an offspring to maintain diversity.	Randomly change the position of one queen.
Generation	One complete cycle of selection, crossover, and mutation.	Repeat for N generations until solution found.

Steps of a Genetic Algorithm:

Step	Action	Description
1	**Initialize**	Randomly generate an initial population of P individuals (candidate solutions).
2	**Evaluate Fitness**	Compute the fitness value for each individual using the fitness function.
3	**Termination?**	If termination criterion met (max generations, fitness threshold reached) → STOP.
4	**Selection**	Select pairs of parents for reproduction. Fitter individuals have higher selection probability.
5	**Crossover**	Combine parent chromosomes at a random crossover point to produce two offspring.
6	**Mutation**	With small probability p_m, randomly alter one or more genes in the offspring.
7	**New Generation**	The offspring replace the least-fit individuals in the population. Go to Step 2.

Example — 8-Queens with GA:

Step	Example
Encoding	Position of queen in each column: chromosome = [3,1,6,2,8,5,7,4]
Fitness	Count non-attacking pairs: max = 28. fitness([3,1,6,2,8,5,7,4]) = 24 (4 conflicts)
Crossover	Parent1:[3,1,6,2,8,5,7,4] Parent2:[4,6,1,5,7,2,3,8] → at point 3: Child:[3,1,6,5,7,2,3,8]
Mutation	Randomly change position 7 from 3→6: [3,1,6,5,7,2,6,8]
Selection	Higher-fitness chromosomes selected more often for reproduction.

Feature	Hill Climbing	Genetic Algorithm
Population	Single current state (population = 1).	Multiple states simultaneously (population = P).
Operators	Incremental local moves.	Crossover (global) + Mutation (local).
Diversity	Low — single trajectory.	High — maintains diverse population.
Parallelism	Serial (one state at a time).	Inherently parallel (population-based).
Local Optima	Often gets stuck.	Better at escaping — crossover is a global operation.
Best For	Simple, unimodal landscapes.	Complex, multimodal optimization landscapes.

SECTION 5: Adversarial Search — Game Playing

Exam Ref: 2082 Kartik, 2081 Chaitra, 2081 Ashwin, 2080 Ashwin, 2079 Chaitra, 2078 Chaitra, 2078 Baishakh, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra, 2074 Bhadra, 2073 Bhadra, 2072 Ashwin, 2071 Bhadra, 2070 Magh

Q11. What is adversarial search? Explain the Minimax algorithm with algorithm steps, an example game tree, and its properties.

[2+6]
2081 Chaitra 2079 Chaitra 2076 Baisakh

Answer: Adversarial Search and Minimax

Adversarial search deals with problems where two or more agents with conflicting goals compete — each tries to maximize their own outcome at the expense of the other. These are game-playing problems. The most common model is a two-player, zero-sum, perfect information game.

Properties of Two-Player Zero-Sum Games:

- Two players: MAX (wants to maximize the utility/score) and MIN (wants to minimize it).
- Zero-sum: What MAX gains, MIN loses. Total utility is constant.
- Perfect information: Both players see the entire board state (Chess, Tic-Tac-Toe, Go).
- Terminal states: States where the game is over (win, lose, draw).
- Utility function: Assigns a value to terminal states (e.g., +1 = MAX wins, −1 = MIN wins, 0 = draw).

Minimax Algorithm:

Minimax performs a depth-first search of the game tree, assuming both players play optimally. MAX always chooses the move with the highest minimax value; MIN always chooses the lowest.

Step	Action	Description
1	**Expand Tree**	Build the complete game tree from current state to all terminal states (or to a depth limit).
2	**Evaluate Leaves**	Apply the utility function to all terminal (leaf) states.
3	**Back-propagate MAX**	At MAX nodes (MAX's turn): assign the MAXIMUM value of all child nodes.
4	**Back-propagate MIN**	At MIN nodes (MIN's turn): assign the MINIMUM value of all child nodes.
5	**Root Decision**	MAX chooses the action leading to the child with the highest minimax value.

Minimax Example — Tic-Tac-Toe / Game Tree:

Consider this simplified 2-ply game tree (MAX moves first, then MIN, then terminal):

Level	Node	Children Values	Minimax Value	Rule Applied
Terminal	Leaf nodes	3, 12, 8	3, 12, 8	Utility function
MIN (depth 1)	B, C, D	B has [3,12], C has [8], D has [2,4,6]	B=3, C=8, D=2	MIN of children
MAX (root)	A	B=3, C=8, D=2	8	MAX of children

MAX picks action leading to node C (value = 8) as the best move.

Property	Value
Completeness	YES — in a finite game tree.
Optimality	YES — optimal against a perfect (rational) opponent.

Time Complexity	O(b^m) — b=branching factor, m=max depth. Chess: b≈35, m≈100 → 35^100 (infeasible).
Space Complexity	O(b×m) — DFS traversal, linear space.
Limitation	Entire game tree must be explored — exponential time. Impractical for real games like Chess/Go.

Q12. What is Alpha-Beta Pruning? Explain its algorithm with example. How does it improve upon the standard Minimax algorithm?

[2+6]
2080 Ashwin 2078 Baishakh 2074 Bhadra

Answer: Alpha-Beta Pruning

Alpha-Beta Pruning is an optimization of the Minimax algorithm that eliminates branches of the game tree that cannot influence the final decision. It produces the SAME result as full Minimax but explores significantly fewer nodes.

Alpha-Beta Parameters:

Parameter	Meaning	Maintained By
Alpha (α)	The best (highest) value that MAX has found so far along the current path. Initially α = −∞.	MAX nodes — updated when a better MAX value is found.
Beta (β)	The best (lowest) value that MIN has found so far along the current path. Initially β = +∞.	MIN nodes — updated when a better MIN value is found.
Pruning Condition	If α ≥ β at any node, cut off the remaining children of that node.	Both levels — Alpha cutoff at MIN; Beta cutoff at MAX.

Step	Action	Description
1	**Initialize**	Start search at root with α = −∞, β = +∞.
2	**MAX Node**	For each child: recurse with current α, β. Update α = max(α, child_value).
3	**Alpha Cutoff**	At MAX node: if α ≥ β — prune remaining children (MIN won't choose this path anyway). Return α.
4	**MIN Node**	For each child: recurse with current α, β. Update β = min(β, child_value).
5	**Beta Cutoff**	At MIN node: if α ≥ β — prune remaining children (MAX already has a better option). Return β.
6	**Result**	Return the minimax value at the root — identical to full Minimax result.

Alpha-Beta vs Full Minimax — Comparison:

Feature	Full Minimax	Alpha-Beta Pruning
Correctness	Correct — explores all nodes.	Correct — same result as Minimax.
Nodes Explored	All nodes: O(b^m).	Best case: O(b^(m/2)) — can explore TWICE as deep!
Pruning	None.	Alpha cutoff (at MIN nodes) + Beta cutoff (at MAX nodes).
Node Order Effect	Order doesn't matter.	Best-first ordering maximizes pruning efficiency.
Best Case	—	Explores only b^(m/2) nodes (when best moves explored first).
Worst Case	O(b^m)	O(b^m) — no pruning when worst moves explored first.
Practical Speedup	—	~250× speedup for Chess — enables searching ~8 plies instead of 4.

Key insight: Alpha-Beta achieves the same result as Minimax but by recognizing that certain subtrees cannot possibly affect the final decision of a rational player — they can safely be "pruned" (ignored).

Most Repeated: "Explain Minimax algorithm and Alpha-Beta pruning with game tree example." This question appears almost every year. Draw a tree with at least 3 levels, show α and β values at each node, and indicate which branches are pruned.

Q13. Explain the concept of game playing in AI. What are the different types of games? What limitations does Minimax face for complex games like Chess, and how are they addressed?

[2+4+2]
2079 Chaitra 2077 Chaitra 2073 Bhadra

Answer: Game Playing in AI

Game playing is a classic area of AI research where intelligent agents must make decisions in competitive, adversarial environments. It tests planning, reasoning, and strategic thinking.

Types of Games:

Dimension	Type 1	Type 2	Examples
Information	Perfect Information (both players see everything)	Imperfect Information (hidden info)	Chess, Go vs Poker, Bridge
Chance	Deterministic (no randomness)	Stochastic (dice, chance)	Chess vs Backgammon, Monopoly
Players	Two-player	Multi-player	Chess vs Poker (4 players)

Sum	Zero-sum (win/loss balanced)	Non-zero-sum (cooperative possible)	Chess vs Diplomacy

Limitations of Minimax for Complex Games (e.g., Chess):

Problem	Explanation	Solution
Exponential time	Chess: b≈35, m≈100. 35^100 states — impossible to evaluate all.	Alpha-Beta pruning + depth limit.
Depth limit needed	Cannot search to terminal — game too long.	Use heuristic evaluation function at cutoff depth (e-value).
Horizon Effect	A disastrous move just below search depth is missed.	Quiescence search — extend until stable position.
Evaluation imprecision	Heuristic e-values are approximations — can mislead search.	Better hand-crafted or learned evaluation functions.
Opening/Endgame	Special patterns not captured by general search.	Opening books (precomputed) + endgame tablebases.

Practical Enhancements to Game Search:

- Heuristic Evaluation Function: e(n) ≈ h*(n). For Chess: material count + positional factors.
- Transposition Table: Cache previously evaluated positions to avoid recomputing.
- Iterative Deepening in Game Trees: Search depth 1, 2, 3, ... and use time budget.
- Monte Carlo Tree Search (MCTS): Used in AlphaGo — simulates random games (playouts) and chooses statistically best move.
- Neural Networks: AlphaZero uses deep neural nets to replace handcrafted evaluation functions — learned entirely from self-play.

SECTION 6: Constraint Satisfaction Problems & Additional Topics

Exam Ref: 2082 Kartik, 2081 Ashwin, 2080 Ashwin, 2079 Ashwin Back, 2078 Chaitra, 2077 Chaitra, 2076 Baisakh, 2075 Bhadra

Q14. What is a Constraint Satisfaction Problem (CSP)? Define its components. Explain Backtracking Search for CSP with an example.

[2+5+1]
2081 Ashwin 2078 Chaitra 2076 Baisakh

Answer: Constraint Satisfaction Problems (CSP)

A Constraint Satisfaction Problem (CSP) is a special type of search problem where the solution is defined by assigning values to variables such that all constraints are satisfied. CSPs exploit the structure of the problem for more efficient search.

Component	Definition	Example (Map Coloring)
Variables X = {X1, X2, ..., Xn}	A set of variables to be assigned values.	Colors of regions: WA, NT, SA, Q, NSW, V, T
Domains D = {D1, D2, ..., Dn}	For each variable Xi, a domain Di of possible values.	Domain = {Red, Green, Blue} for each region
Constraints C	Pairs (scope, relation): which value combinations are allowed.	Adjacent regions must have DIFFERENT colors.
Goal	Find a complete assignment satisfying ALL constraints.	Assign colors so no two neighboring regions share a color.

Types of Constraints:

- Unary Constraint: Restricts a single variable. Example: SA ≠ Green.
- Binary Constraint: Relates two variables. Example: WA ≠ NT. Most CSPs reduce to binary constraints.
- Global Constraint: Involves more than 2 variables. Example: AllDiff(Q1,Q2,...,Q8) in N-Queens.

Backtracking Search for CSP:

Backtracking is the standard algorithm for CSPs. It assigns one variable at a time and backtracks when no valid value is found for a variable (constraint violated).

Step	Action	Description
1	**Select Variable**	Choose an unassigned variable (e.g., using Minimum Remaining Values — MRV heuristic).
2	**Order Values**	Order values in domain (e.g., Least Constraining Value — LCV heuristic).
3	**Assign**	Try assigning the next value to the selected variable.
4	**Check Constraints**	If the assignment is consistent (no constraint violated), continue recursively.
5	**Recurse**	Recursively assign the next variable. If successful, return the complete assignment.
6	**Backtrack**	If no value works for the current variable, UNDO the last assignment and try the next value for the previous variable.

CSP Heuristics (Improve Backtracking Efficiency):

Heuristic	Description	Benefit
MRV (Minimum Remaining Values)	Choose the variable with the FEWEST legal values remaining in its domain first. Also called "fail-first."	Catches constraint violations early — reduces search space.
Degree Heuristic	Among variables with same domain size, choose the one involved in the MOST constraints with other unassigned variables.	Used as tiebreaker for MRV.
LCV (Least Constraining Value)	Choose the value that rules out the fewest choices for neighboring unassigned variables.	Maximizes flexibility for remaining assignments.
Forward Checking	After each assignment, check remaining variables' domains. Remove values that violate constraints.	Detects dead ends early, before they occur.
Arc Consistency (AC-3)	Enforce arc consistency across all variable pairs. Propagate constraint removal throughout the graph.	Reduces domains significantly before search.

Q15. Compare different search strategies. Given a search tree or graph, trace through BFS, DFS, and A* to find the goal and determine the optimal path.

[4+4]
2082 Kartik 2080
Ashwin 2078 Baishakh

Answer: Comparative Search Trace Example

Consider the following graph for search (nodes with edges and costs). Goal: Find path from A to G.

	Nodes	Edges (cost)	Heuristic h(n) to G
Graph	A,B,C,D,E,F,G	A-B(1),A-C(4),B-D(2),B-E(5),C-F(3),D-G(3),E-G(1),F-G(2)	h(A)=6,h(B)=4,h(C)=3,h(D)=2,h(E)=1,h(F)=2,h(G)=0

BFS Trace (ignoring costs):

Step	Queue (FIFO)	Visited	Expanded
1	[A]	{}	A → children: B, C
2	[B, C]	{A}	B → children: D, E
3	[C, D, E]	{A,B}	C → children: F
4	[D, E, F]	{A,B,C}	D → children: G
5	[E, F, G]	{A,B,C,D}	G = GOAL!

- BFS Path: A → B → D → G | BFS Cost: 1+2+3 = 6 (not optimal — BFS ignores costs)

A* Trace (f = g + h):

Step	Priority Queue (by f)	Node Expanded	f = g + h
1	[(A,0+6=6)]	A	f(A)=6
2	[(B,1+4=5),(C,4+3=7)]	B	f(B)=5
3	[(D,3+2=5),(C,4+3=7),(E,6+1=7)]	D	f(D)=5
4	[(G,6+0=6),(C,4+3=7),(E,6+1=7)]	G	f(G)=6 = GOAL!

- A* Path: A → B → D → G | A* Cost: 1+2+3 = 6 (optimal)

Comparison Summary for this example:

Algorithm	Path Found	Total Cost	Nodes Expanded	Optimal?
BFS	A→B→D→G	6 (by luck same)	A,B,C,D,G = 5	Not guaranteed
DFS	A→B→D→G or A→B→E→G	6 or 6	Depends on order	No
A* (h=Manhattan)	A→B→D→G	6	A,B,D,G = 4	YES

QUICK REVISION: Chapter 3 — Key Points Summary

Topic	Key Points	Expected Marks
Problem Formulation	5 components: Initial State, Actions, Transition Model, Goal Test, Path Cost. 8-puzzle = 9!/2 = 181,440 reachable states.	2-4
BFS	FIFO queue; Complete + Optimal* (equal costs); Time & Space O(b^d). Use when goal is shallow.	4-6
DFS	LIFO stack; NOT complete (infinite), NOT optimal; Time O(b^m), Space O(b·m). Advantage: low memory.	4-6
UCS	Priority queue by g(n); Complete + Optimal (any cost); O(b^C*/ε). Use for varying step costs.	2-4
DLS / IDDFS	DLS adds depth cutoff L to DFS. IDDFS runs DLS with L=0,1,2,...; Complete+Optimal+low memory O(b·d).	2-4

Heuristic / A*	h(n)=estimate to goal; f(n)=g(n)+h(n); Admissible h → A* optimal; h2 (Manhattan) dominates h1 (misplaced).	4-8
Greedy vs A*	Greedy: f=h(n), fast but not optimal/complete. A*: f=g+h, optimal if h admissible.	2-4
Hill Climbing	Single state; move to best neighbor; Stuck at local max/plateau/ridge. Fix: random restart, SA, GA.	2-4
Simulated Annealing	Accept bad moves with P=e^(ΔE/T); high T=random; T→0=hill climbing. Theoretically finds global optimum.	2-4
Genetic Algorithm	Population + fitness + selection + crossover + mutation. Escapes local optima via crossover.	2-4
Minimax	MAX picks max; MIN picks min; Complete + Optimal vs rational opponent; O(b^m) time.	4-8
Alpha-Beta Pruning	Same result as Minimax; prune if α≥β; best case O(b^m/2). Alpha=best MAX; Beta=best MIN.	4-8
CSP	Variables + Domains + Constraints; Backtracking + MRV + LCV + Forward Checking = efficient CSP solver.	2-4

Best of Luck! | TU IOE BCT | CT 653 AI | Chapter 3: Solving Problems by Searching